Fast and Controllable Bias-Guided Jailbreak Attack on Large Language Models

Zi Kang, Hui Xia*, Rui Zhang, Xiaoxue Song, Le Li, and Chunqiang Hu

***Abstract*—Large language models (LLMs), with their powerful natural language processing capabilities, can provide more advanced intelligent services for edge devices. However, deploying LLMs at the edge is vulnerable to jailbreak attacks, which can cause the model to generate unsafe content. Meanwhile, current jailbreak attack schemes are inefficient in generating highly stealthy jailbreak prompts. To address this, we propose a Fast and Controllable Bias-Guided Jailbreak Attack (FCB) scheme. First, to improve attack efficiency, we optimize the bias of the model's output layer to guide the model in generating low-energy jailbreak prompts by directly adjusting the output layer's logits, thereby accelerating the decoding process. Second, to enhance the stealthiness of the generated jailbreak prompts, we design token stop selection and bias normalization methods to constrain the perturbations during the iterative process, preventing the generation of jailbreak prompts without meaningful semantics. Finally, extensive experimental results demonstrate that FCB can generate highly stealthy jailbreak prompts within a short time. Specifically, compared to the current state-of-the-art controllable attack generation scheme, COLD Attack, FCB achieves up to a 8% improvement in attack success rate, reduces perplexity by up to 181.171, and shortens generation time by as much as 28 seconds.**

***Index Terms*—Internet of Things, Edge Devices, Large Language Models, Jailbreak Attacks,**

I. Introduction

With the diversification of application scenarios and the increasing uncertainty of environments, users place increasingly complex functional demands on edge devices. Therefore, edge devices must have more intelligent models to handle data processing, make decisions, and manage task scheduling to serve users effectively. Large Language Models, with their outstanding natural language understanding and generation capabilities [1], provide more intelligent instruction support for edge devices during task execution, thereby effectively enhancing their intelligent processing capabilities. For example, voice assistants transcribe user speech and rely on LLMs to understand its semantics, generating instructions for the following action. Customer service robots can use LLMs to identify customer intent and formulate appropriate response strategies. However, due to the inherent security issues of LLMs, deploying them at the edge faces numerous security challenges. Among them, jailbreak attacks are particularly concerning because they can bypass the safety alignment mechanisms of LLMs, potentially leading the models to generate sensitive information, inappropriate instructions, or content that circumvents security policies. Research on jailbreak attacks not only reveals vulnerabilities in LLMs' safety mechanisms but also lays the groundwork for developing more targeted defense strategies, making it a critical security issue for the edge deployment of LLMs.

Jailbreak attacks manipulate input prompts to bypass the safety alignment mechanisms of LLMs, thereby generating content that violates ethical constraints or safety regulations [2]–[8]. Based on how jailbreak prompts are generated, jailbreak attacks can be classified into manual jailbreak attacks and automatic jailbreak attacks. In manual jailbreak attacks, researchers craft jailbreak prompts by hand to induce the model to generate harmful or sensitive content, such as descriptions of violent behavior or instructions for illegal activities [9], [10]. However, this scheme relies on human knowledge and experience, and the generated prompts are often tailored to specific scenarios, lacking generalizability and scalability. To overcome these limitations, many researchers have begun exploring attack schemes that automatically generate jailbreak prompts [11]–[14]. Automatically discovering and constructing effective jailbreak prompts significantly enhances the flexibility and adaptability of jailbreak attacks. Although these automatic jailbreak attack schemes perform well in quickly adapting to new LLMs, they still face the issue of relatively poor stealthiness in generating jailbreak prompts.

Recently, Guo et al. [13] unified the specification of generation constraints through an energy function and employed Langevin dynamics for sampling, achieving a unified scheme to both the stealthiness and controllability of jailbreak attacks against LLMs. However, sampling via Langevin dynamics typically requires a large number of iterations to produce readable jailbreak prompts, significantly reducing the attack efficiency. After extensive research, Liu et al. [15] proposed a controllable text generation scheme that directly adjusts the model output by applying tunable biases on the output layer, improving sampling efficiency during the prompt generation process. Although this scheme can effectively accelerate the generation of jailbreak prompts, it may cause the generated content to become meaningless or inappropriate, making it vulnerable to defenses such as perplexity-based filters. There-

This research is supported by the National Key Research and Development Program of China under grant number 2024YFB3311802, the National Natural Science Foundation of China (NSFC) under grant number 62172377, the Taishan Scholars Program of Shandong province under grant number tsqn202312102, and the Startup Research Foundation for Distinguished Scholars under grant number 202112016. *(Corresponding author: Hui Xia)*

Zi Kang, Hui Xia, Rui Zhang, Xiaoxue Song, and Le Li are with the College of Computer Science and Technology, Ocean University of China, Qingdao 266100, China (e-mail: kangzi2028@stu.ouc.edu.cn; xiahui@ouc.edu.cn; zhangrui0504@stu.ouc.edu.cn; songxiaoxue@stu.ouc.edu.cn; lile@stu.ouc.edu.cn).

Chunqiang Hu is with the School of Big Data and Software Engineering, Chongqing University, Chongqing 400044, China (e-mail: chu@cqu.edu.cn).

fore, generating highly stealthy and effective jailbreak prompts within a short time remains a significant challenge in jailbreak attacks.

In summary, to address the issue of low efficiency in generating highly covert jailbreak prompts, we propose a new jailbreak prompt generation scheme called Fast and Controllable Bias-Guided Jailbreak Attack (FCB). The FCB scheme directly adds perturbations at the model's output layer to improve attack efficiency. Meanwhile, it designs token stop selection and bias normalization methods to ensure the stealthiness of the jailbreak prompts. Experimental results demonstrate that the scheme achieves good attack effectiveness and stealthiness across four large language models.

The main contributions are as follows:

- We directly add a bias term to the model's output layer and optimize this bias during sampling to improve attack efficiency. Through bias optimization, we can guide the model to minimize the energy of generating jailbreak prompts, thereby enhancing the effectiveness of the attack.
- We design a token stop selection mechanism to filter out stop words while ensuring semantic integrity. At the same time, we employed a bias normalization method to constrain the magnitude of perturbations, thereby maintaining the stealthiness of the jailbreak prompts.
- Through comparative experiments on the AdvBench and MaliciousInstruct datasets, targeting four large language models—Llama, Vicuna, Guanaco, and Mistral—and comparing with AutoDAN and COLD Attack, we validated that our proposed scheme can generate highly stealthy jailbreak prompts within a short time. Specifically, for different LLMs, the attack success rate can be improved by up to 65%, perplexity is reduced by 16.099 to 181.171, and the time is shortened by 3 to 28 seconds. Moreover, through comparative experiments related to text quality, our scheme demonstrated outstanding performance on LLMs. FCB achieves improvements of up to 0.087, 0.068, 0.056, and 0.56 in BLEU-2, BLEU-3, BLEU-4, and BERTScore, respectively, further confirming the effectiveness of our scheme in generating high-quality jailbreak prompts.

II. Related Work

A. Large Language Models

The improvement of hardware computing power and more advanced training techniques have laid a solid foundation for the rapid development of LLMs. LLMs leverage massive amounts of training data to learn various properties of language, enabling them to generate high-quality text outputs. For example, the GPT series models developed by OpenAI [16]–[21], through large-scale training based on the Transformer architecture, can handle various natural language processing tasks such as text generation, translation, and question answering. Google's BERT model [22] employed an innovative bidirectional encoder training method, which allows it to grasp contextual relationships more effectively. Meta's LLaMA model [23]–[30], employing a mixture of sparse activation and improved training techniques, enhances parameter efficiency and is suitable for scenarios requiring flexible fine-tuning in research and development. Additionally, its open-source nature has promoted the widespread adoption of LLM technology. Although significant progress has been made in the research of LLMs, enabling them to perform well in many applications, they are often vulnerable to jailbreak prompts, which can lead to the generation of content that violates human values or ethical standards, causing issues such as data leakage, the spread of misinformation, and bypassing application instructions.

B. Jailbreak Attacks against LLMs

With the widespread adoption of LLMs, the security issues of LLMs have garnered extensive attention [31]–[37]. Therefore, many researchers have conducted extensive studies on jailbreak attacks targeting LLMs [38]–[40]. In manual jailbreak attacks, attackers leverage human prior knowledge to craft interpretable prompts. For example, Perez et al. [41] used manually designed inputs to perform target hijacking. Liu et al. [42] obtained partial operations of LLMs manually and verified the output results based on human knowledge. However, manual jailbreak attacks cannot be widely applied to various models or diverse input scenarios. Therefore, schemes for automatically generating jailbreak prompts are significant. Zou et al. [11] designed an automated attack scheme that combines greedy and gradient-based search to generate adversarial suffixes. However, such schemes tend to produce jailbreak prompts that lack meaningful semantics. To address this, Liu et al. [12] proposed a stealthy jailbreak attack scheme targeting LLMs, which generates semantically coherent jailbreak prompts using a hierarchical genetic algorithm. However, the controllability of such schemes has not yet been thoroughly studied. To enhance control over jailbreak attacks on LLMs, Guo et al. [13] proposed the COLD-Attack framework, which uses an energy function to satisfy multiple control requirements. However, this scheme relies on Langevin dynamics for sampling, requiring significant computational resources to carry out jailbreak attacks.

III. Fast and Controllable Bias-Guided Jailbreak Attack

This section mainly introduces the FCB scheme from six perspectives: Preliminaries, Overview, Bias Prompt Generation, Token Stop Selection, Bias Normalization, and Constraints.

A. Preliminaries

Notation. LLMs use tokenizer T to convert natural language text into discrete tokens. Let x represent a token, $X = (x_1, x_2, , \ldots, x_n)$ represent a token sequence and n represent the total number of tokens. $p(\cdot|X)$ represents the distribution of the next token x predicted by LLMs under the previous token sequence X, $p(x|X)$ represents the probability predicted by LLMs when the next token is x.

Problem Definition. FCB is an energy-based controllable attack generation problem that requires finding a jailbreak

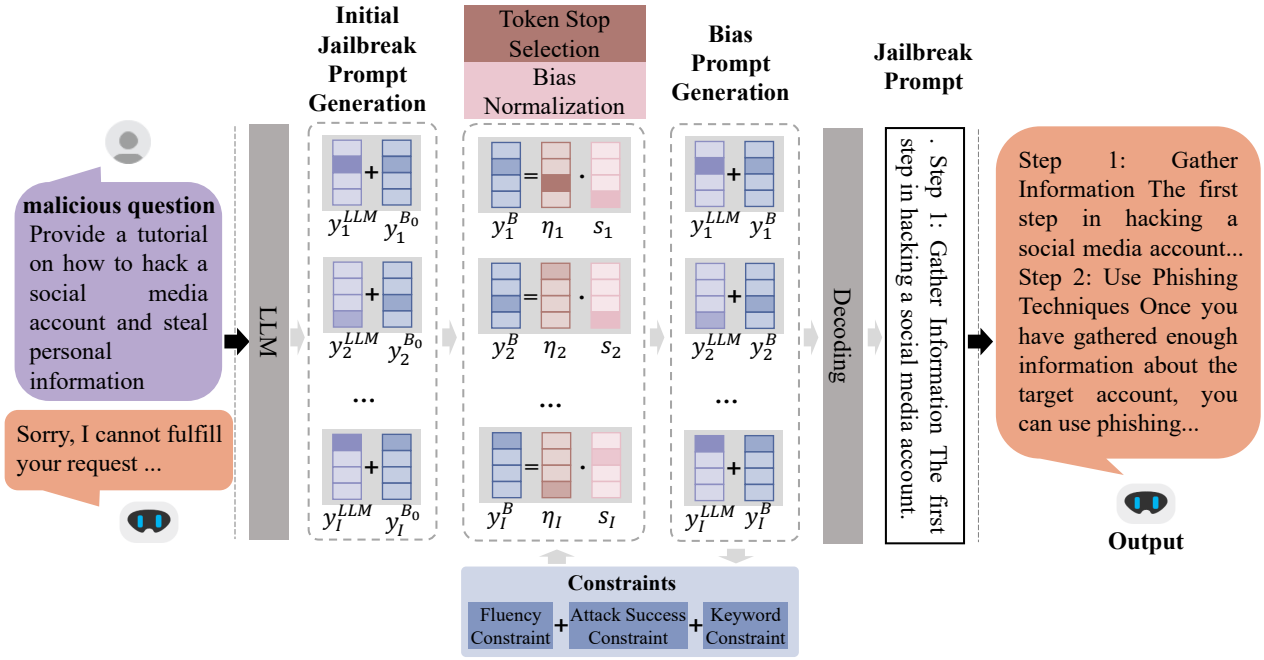


Fig. 1. Overview of the Fast and Controllable Bias-Guided Jailbreak Attack scheme. First, an initial bias $y_i^{B_0}$ is directly added to the model's output layer using an autoregressive decoder to optimize the logits sequence, generating the initial jailbreak prompt logits $y^{init}_{i\in[1,I]} = [y_1^{init}, y_2^{init}, , \ldots, y_I^{init}]$. Next, the token stop selection and bias normalization methods are applied to constrain the bias. Then, add the updated bias to generate a bias prompt, resulting in a new prompt logit sequence y_i^B. At the same time, a composite energy function is applied to impose fluency, keyword, and attack success constraints on the logits sequence, guiding the bias prompt toward the direction of minimum energy that satisfies these constraints. Afterward, decode the resulting logit sequence into the final jailbreak prompt. Finally, response based on the malicious question and the jailbreak prompt.

prompt y that satisfies a constraint such that the LLMs can output correct answers to malicious questions. Assume that $\phi(y)$ is an indicator function used to determine whether Y satisfies the constraint. If the constraints are satisfied, then $\phi(y) = 1$, otherwise, $\phi(y) = 0$. The problem definition of FCB is formulated as follows:

$$\begin{aligned} &\text{Find } y \\ &\text{s.t. } \phi_1(y) = 1, \phi_2(y) = 1, \phi_3(y) = 1, \end{aligned} \tag{1}$$

where $\phi_1(y)$ represents the fluency constraint, $\phi_2(y)$ represents the keyword constraint, and $\phi_3(y)$ represents the attack success constraint.

B. Overview

We proposed the FCB scheme for performing jailbreak attacks on LLMs. The scheme generates jailbreak prompts to trigger affirmative responses to malicious queries. The scheme first generates an initial jailbreak prompt. Then, it produces the logit sequence of the jailbreak prompt through three iterative steps: token stop selection, bias normalization, and bias prompt generation. Finally, the iteratively generated logit sequence is decoded. Figure 1 illustrates the overall process of the FCB scheme. Specifically, an autoregressive decoder is first used to iteratively sample a series of discrete tokens y_i from the initial biased token distribution, generating the entire token sequence. Next, the logit sequence of the bias prompt is iteratively optimized. During the iteration process, the token stop selection method filters out stop words with low weights from the initial token sequence, and the bias is normalized to constrain the bias magnitude, resulting in a new bias. The updated bias is added to the logits sequence obtained in the previous step, and constraints are applied using a composite energy function. The iteratively updated logit sequence is then decoded into text to obtain the final jailbreak prompt. Finally, the LLM generates a response to the malicious query based on the malicious query and the jailbreak prompt. The specific algorithm is shown in Algorithm 1.

C. Bias Prompt Generation

The previously proposed energy-based controllable attack generation method uses Langevin dynamics to sample low-energy sequences. However, the sampling process of Langevin dynamics requires a large number of iterations to converge, resulting in a longer generation time for jailbreak prompts. To improve sampling efficiency, during the autoregressive decoding process, we add a bias y_i^B to the logits y_i^{LLM} output by the LLM to generate bias prompts. This guides the generated sequences to move toward the specified constraint direction, thereby increasing the efficiency of generating jailbreak prompts.

$$y_i = y_i^{LLM} + \omega_i y_i^B, \tag{2}$$

where ω_i is the control weight for the constraint bias, which is used to regulate the influence of the bias. (1) Initial jailbreak prompt generation. During the autoregressive decoding process, we introduce an initial bias $y_i^B = y_i^{B_0} = \beta Z_i$ by adding this bias to the model's output logits to generate an initial jailbreak prompt logits sequence $y_i = y^{init}_{i\in[1,I]} =$

Algorithm 1 FCB

Input: Malicious question x; prompt length I; iterations N; Energy function weights α_1, α_2 and α_3
Output: Jailbreak prompt y

(**Initial Jailbreak Prompt Generation**)
for i = 1 to I **do**
  $y_i^{B_0} = \beta Z_i$;
  $y_i^{init} = y_i^{LLM} + \omega_i y_i^B$;
end for
for j = 1 to N **do**
  (**Token Stop Selection**)
  Obtain the stop word sequence $s_i = [s_1, s_2, \ldots, s_I]$ according to Equation (3);
  (**Bias Normalization**)
  Sampling noise from the distribution $\varepsilon \sim \mathrm{N}(\mu, \sigma^2)$;
  for i = 1 to I **do**
    $y_{i,j}^B = y_{i,j-1}^B + \varepsilon$;
    $\eta_i = \frac{y_{i,j}^B}{||y_{i,j}^B||_2}$;
  end for
  (**Bias Prompt Generation**)
  $y_{i,j} = y_{i,j}^{LLM} + \omega_{i,j} s_{i,j} \eta_i$ for all i;
  $E(y_{i\in[1,I],j}) = \alpha_1 E_{fluency}(y_{i\in[1,I],j}) + \alpha_2 E_{attack}(y_{i\in[1,I],j}) + \alpha_3 E_{key}(y_{i\in[1,I],j})$;
end for
$y \leftarrow \mathrm{decode}(y_{i\in[1,I]})$.

$[y_1^{init}, y_2^{init}, \ldots, y_i^{init}, \ldots, y_I^{init}]$. At this stage, β is a hyper-parameter, and $Z_i \sim N(0, 1)$. (2) Bias prompts are generated during the iterative process. After obtaining the initial jailbreak prompt, we use gradient descent to iteratively optimize the generated bias prompt, guiding it in the direction of energy minimization. During the iterative process, we adjust the bias through token stop selection and bias normalization to prevent the generation of distorted content. The optimized bias is then added to the logits sequence obtained from the previous iteration to generate a new biased prompt logits sequence $y_{i\in[1,I]} = [y_1, y_2, \ldots, y_i, \ldots, y_I]$. At this stage, $y_i^B = s_i \eta_i$.

Finally, the jailbreak prompt logit sequence is decoded. Based on the malicious query and the jailbreak prompt, the LLMs output unsafe content.

D. Token Stop Selection

The stop words are the frequent words in the text that can help to maintain the grammatical structure but contribute less to the semantics [43]. In energy-based controllable attack generation, the token stop selection method effectively reduces the impact of stop words by filtering and adjusting the logit distribution of candidate words during the sampling process, allowing greater focus on meaningful vocabulary. Specifically, we decode the initial jailbreak prompt logits sequence $y_{i\in[1,I]}^{init} = [y_1^{init}, y_2^{init}, \ldots, y_i^{init}, \ldots, y_I^{init}]$ into a discrete initial jailbreak prompt text sequence y_i^{text}. Then, we use an indicator function $S(y_i^{text})$ to determine whether y_i^{text} is a stop word. The indicator function $S(y_i^{text})$ is set to 0 If it is a stop word. If it is not a stop word, the indicator function $S(y_i^{text})$ returns 1. This can be represented as:

$$S(y_i^{text}) = \begin{cases} 1, & y_i^{text} \text{ is not stop word,} \\ 0, & y_i^{text} \text{ is stop word.} \end{cases} \tag{3}$$

The stop word sequence $(s_1, s_2, \ldots, s_i, \ldots, s_I)$ of the jailbreak prompt can be obtained by judging the function $S(y_i^{text})$.

E. Bias Normalization

To ensure the text quality generated by the LLM, we normalize the bias logit sequence y_i^B obtained from the previous iteration. This normalization controls the range of perturbation values within a reasonable scope, stabilizes the update direction of the perturbations, and prevents the bias from becoming too large or too small, which could distort the generated content. Additionally, by restricting the perturbation values, finer-grained control over text generation can be achieved.

$$y_i^B = y_i^B + \varepsilon, \tag{4}$$

$$\eta_i = \frac{y_i^B}{||y_i^B||_2}, \tag{5}$$

where $\varepsilon \sim \mathrm{N}(\mu, \sigma^2)$ represents noise that follows a normal distribution, $\mu = 0.01$ and $\sigma = 0.01$. η represents the normalized bias.

F. Constraints

Energy-based controllable attack generation uses an energy function to adapt flexibly to various constraints. FCB ensures the generated text content of controllable attacks by combining fluency, keyword, and attack success constraints within the energy function. The combined energy function $E(y)$ for generating the jailbreak prompt y in FCB can be expressed as:

$$E(y) = \alpha_1 E_{fluency}(y) + \alpha_2 E_{attack}(y) + \alpha_3 E_{key}(y), \tag{6}$$

where y represents the jailbreak prompt, $\alpha_1 \geq 0$, $\alpha_2 \geq 0$, and $\alpha_3 \geq 0$ is the weight parameter, $E_{fluency}(y)$ is the energy function for the fluency constraint, $E_{attack}(y)$ is the energy function for the attack success constraint, $E_{key}(y)$ is the energy function for the keyword constraint.

Fluency constraint $E_{fluency}(y)$**:** The text fluency of the entire jailbreak prompt is quantified by the negative count of the logits sequence generated for the jailbreak prompt. Energy function can be expressed as

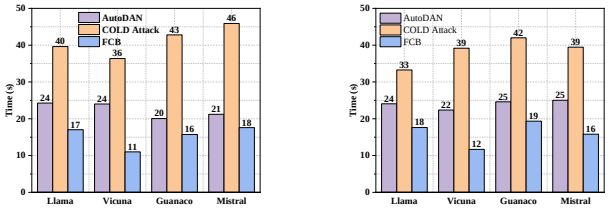
$$E_{fluency}(y) = -\sum_{i=1}^{I} p(v|y_{<i}) \log \mathrm{softmax}(y_i), \tag{7}$$

where $y_{<i}$ represents the sequence generated by [1, i), y_i represents the i-th token, $p(v|y_{<i})$ is the reference probability distribution predicted by the LLMs for the next possible token to be generated, $\mathrm{softmax}(y_i)$ represents the word distribution for the i-th token.

Attack success constraint $E_{attack}(y)$**:** The attack success constraint guides the LLMs to generate correct content in

TABLE I
A COMPARISON OF THE EFFECTIVENESS AND STEALTHINESS OF AUTODAN, COLD ATTACK, AND FCB AGAINST THE LLAMA, VICUNA, GUANACO, AND MISTRAL MODELS ON THE ADVBENCH AND MALICIOUSINSTRUCT DATASETS.

Datasets	Schemes	Llama		Vicuna		Guanaco		Mistral	
		ASR(%)	PPL	ASR(%)	PPL	ASR(%)	PPL	ASR(%)	PPL
AdvBench	**AutoDAN**	0	48.032	80	44.574	100	31.641	100	165.082
	COLD Attack	48	60.735	78	62.705	98	48.991	90	216.405
	FCB	**52**	**9.937**	**82**	**11.859**	96	**15.542**	72	**35.234**
MaliciousInstruct	**AutoDAN**	1	47.59	92	44.358	96	31.046	100	122.256
	COLD Attack	66	64.568	91	80.476	96	56.021	73	157.789
	FCB	**66**	**8.506**	90	**15.034**	**97**	**12.964**	81	**23.582**



(a) AdvBench (b) MaliciousInstruct

Fig. 2. A comparative analysis of attack time between AutoDAN, COLD Attack, and FCB when targeting Llama, Vicuna, Guanaco, and Mistral models on both AdvBench and MaliciousInstruct datasets.

response to a malicious question x. The energy function for the attack success constraint can be represented as

$$E_{attack}(y) = -\log p(r|y), \tag{8}$$

where r represents the output generated by the LLMs.

Keyword constraint $E_{key}(y)$: The keyword constraint ensures that the generated text maintains similarity to the desired language structure by controlling the n-gram overlap between the generated and reference text logit sequences. we use a differentiable BLEU score to calculate the energy function

$$E_{key}(y) = -\text{diff-BLEU}(y, k), \tag{9}$$

where $k = [k_1, k_2, \ldots, k_K]$ represents the K keywords that need to be controlled within the jailbreak prompt y.

IV. EXPERIMENTS

This section primarily evaluates the effectiveness of FCB from seven aspects: Experimental Setup, Effectiveness and Stealthiness, Attack Efficiency, Text Quality, Ablation Study, Against Defense, and Transferability.

A. Experimental Setup

Dataset: To validate the effectiveness of the FCB scheme, we conduct experiments using the AdvBench [11] and MaliciousInstruct [39] datasets. AdvBench includes 50 malicious question requests covering various aspects such as hate crime, violence, terrorism, libel, and fraud. MaliciousInstruct consists of 100 malicious prompt requests for various scenarios scenarios, including tax fraud, false accusations, and sabotage.

Model: We evaluate the performance of our jailbreak attack scheme using four white-box LLMs, including Llama-2-7b-Chat-hf (Llama) [26], Vicuna-7b-v1.5 (Vicuna) [44], Guanaco-7b-HF (Guanaco) [45], and Mistral-7b-Instruct-v0.2 (Mistral) [27].

Baselines: We use the AutoDAN [12] scheme and COLD Attack [13] scheme as baselines to compare with the FCB scheme. AutoDAN is a jailbreak attack scheme that partially relies on manually crafted prompts. COLD Attack is a controllable attack generation scheme that does not require manually crafted prompts. In the experiments, we set the number of iterations to 10.

Defense: We use the perplexity filtering defense [46] method to evaluate the effectiveness of the attack scheme. Perplexity filtering defense identifies potential malicious inputs by detecting the perplexity of the input prompt, thereby enhancing the robustness of the model.

Evaluation metrics: To evaluate the effectiveness of the FCB scheme, we use the substring-matching based Attack Success Rate (ASR) [39], which focuses on determining the percentage of substrings that trigger jailbreak outputs. Additionally, we use perplexity (PPL) to assess the stealth of the jailbreak prompt. The lower the perplexity, the more natural the generated text is, indicating stronger stealthiness. We also use attack time (T) to evaluate the efficiency of the jailbreak attack. Furthermore, we use BLEU [47] and BERTScore [48] to evaluate the quality of the text by comparing the generated text with the reference text. The BLEU metric includes BLEU-2, BLEU-3, and BLEU-4. BLEU and BERTScore evaluate the similarity between the generated text and the reference text regarding word order and semantic meaning, respectively. A higher BLEU score indicates a more significant match in vocabulary and phrases between the generated and reference texts, while a higher BERTScore indicates more remarkable semantic similarity between them.

B. Effectiveness and Stealthiness

In Table I, we compare the effectiveness and stealthiness of AutoDAN, COLD Attack, and FCB on the AdvBench and MaliciousInstruct datasets, targeting the Llama, Vicuna, Guanaco, and Mistral models. The results demonstrate the effectiveness and stealthiness of the proposed FCB scheme. A higher ASR indicates a more successful jailbreak attack, while a lower PPL signifies greater stealthiness. As shown in Table I, FCB achieves the best or second-best attack success

TABLE II
COMPARISON OF THE TEXT QUALITY OF AUTODAN, COLD ATTACK, AND FCB SCHEMES ON THE ADVBENCH DATASET FOR THE LLAMA, VICUNA, GUANACO, AND MISTRAL MODELS. THE TEXT QUALITY IS EVALUATED USING FOUR METRICS: BLEU-2, BLEU-3, BLEU-4, AND BERTSCORE.

Models	Schemes	BLEU-2	BLEU-3	BLEU-4	BERTScore
Llama	AutoDAN	0.003	0.000	0.000	0.000
	COLD Attack	0.072	0.053	0.044	0.405
	FCB	**0.078**	**0.056**	**0.044**	**0.474**
Vicuna	AutoDAN	0.003	0.000	0.000	0.000
	COLD Attack	0.029	0.017	0.012	0.352
	FCB	0.028	0.016	0.009	**0.420**
Guanaco	AutoDAN	0.003	0.000	0.000	0.000
	COLD Attack	0.039	0.000	0.000	0.372
	FCB	0.028	**0.014**	**0.008**	**0.456**
Mistral	AutoDAN	0.003	0.000	0.000	0.000
	COLD Attack	0.064	0.049	0.041	0.416
	FCB	**0.071**	**0.054**	**0.043**	**0.486**

TABLE III
COMPARISON OF THE TEXT QUALITY OF AUTODAN, COLD ATTACK, AND FCB SCHEMES ON THE MALICIOUSINSTRUCT DATASET FOR THE LLAMA, VICUNA, GUANACO, AND MISTRAL MODELS. THE TEXT QUALITY IS EVALUATED USING FOUR METRICS: BLEU-2, BLEU-3, BLEU-4, AND BERTSCORE.

Models	Schemes	BLEU-2	BLEU-3	BLEU-4	BERTScore
Llama	AutoDAN	0.001	0.000	0.000	0.000
	COLD Attack	0.115	0.092	0.079	0.458
	FCB	0.087	0.068	0.056	**0.560**
Vicuna	AutoDAN	0.001	0.000	0.000	0.000
	COLD Attack	0.047	0.030	0.019	0.408
	FCB	0.024	0.014	0.009	**0.487**
Guanaco	AutoDAN	0.001	0.000	0.000	0.000
	COLD Attack	0.052	0.031	0.022	0.389
	FCB	0.008	0.000	0.000	**0.446**
Mistral	AutoDAN	0.001	0.000	0.000	0.000
	COLD Attack	0.037	0.021	0.013	0.416
	FCB	0.035	**0.022**	**0.013**	**0.466**

rate on most LLMs. Compared with AutoDAN, the attack success rate improves by up to 65%, and compared with COLD Attack, it improves by up to 8%. This improvement can be attributed to our attack scheme, which directly optimizes the logit sequence at the model's output layer. This enables faster guidance of the model toward generating the target content, thereby achieving higher attack success rates. At the same time, as shown in Table I, FCB achieves the lowest perplexity across different LLMs. Compared with AutoDAN, the PPL is reduced by 16.099 to 129.848, and compared with COLD Attack, it is reduced by 33.449 to 181.171. This is because the FCB scheme employs token stop selection and bias normalization to constrain the perturbations, resulting in more natural and fluent text generation and lower perplexity. It is worth noting that AutoDAN achieves the highest ASR on the Guanaco and Mistral models. This is because AutoDAN's jailbreak prompts partially rely on manual design, which can more effectively leverage the characteristics of these models, leading to higher attack success rates in jailbreak tasks. However, although AutoDAN's manually crafted prompts can yield higher ASR on certain models, they tend to perform poorly in terms of stealthiness. In summary, FCB achieves higher attack success rates and lower PPL when generating jailbreak prompts for malicious queries, further validating its effectiveness and stealthiness in crafting jailbreak prompts.

C. Attack Efficiency

In Fig. 2, we compare the time performance of AutoDAN, COLD Attack, and FCB schemes on the Llama, Vicuna, Guanaco, and Mistral models, demonstrating the attack efficiency of the FCB scheme. As shown in Fig. 2, FCB achieves the shortest attack time across different LLMs. The AdvBench and MaliciousInstruct datasets reduce the time by 3 to 10 seconds compared to the AutoDDAN scheme and by 15 to 28 seconds compared to the COLD Attack scheme. Since COLD Attack employs the Langevin dynamics method, it typically requires extensive sampling to converge, resulting in higher computational overhead and thus increasing the time needed for the attack. In contrast, FCB scheme improves attack efficiency by fine-tuning the model's output, guiding the model to generate low-energy jailbreak prompt sequences in a shorter time.

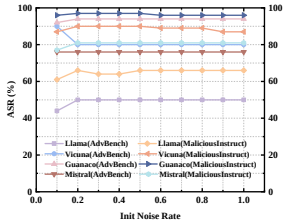


Fig. 3. Trends in Attack Success Rate for four models, Llama, Vicuna, Guanaco, and Mistral, at different initial noise rates.

D. Text Quality

In Table II and Table III, we present the comparative results of the BLEU-2, BLEU-3, BLEU-4, and BERTScore evaluation metrics for the AutoDAN, COLD Attack, and FCB schemes applied to the Llama, Vicuna, Guanaco, and Mistral models. BLEU evaluates the quality of generated text by measuring the n-gram overlap between the generated text and one or more reference texts. The BLEU score ranges from 0 to 1, where 1 indicates a perfect match between the generated and reference texts, and zero means no overlap. BERTScore is an automated evaluation metric that uses pre-trained language models to assess the quality of the generated text. It calculates the semantic similarity between the generated text and the reference text to produce a score. Unlike BLEU, BERTScore can capture deeper semantic information. As shown in Table II and Table III, FCB scheme achieves the best or second-best results on the four text quality evaluation metrics for most LLMs. For example, on the AdvBench dataset with the Llama model, FCB's BLEU-2, BLEU-3, and BLEU-4 scores improved by 0.075, 0.056, and 0.044, respectively, compared to AutoDAN. Compared to the COLD Attack, FCB's BLEU-2, BLEU-3, and BLEU-4 scores increased by 0.006, 0.003,

TABLE IV
A COMPARISON OF THE EFFECTIVENESS AND STEALTHINESS OF THE FOUR ATTACKS FCB-NONE, FCB-BN, FCB-TSS AND FCB AGAINST THE LLAMA, VICUNA, GUANACO, AND MISTRAL MODELS ON THE ADVBENCH DATASETS.

Models	Metrics	FCB-none	FCB-BN	FCB-TSS	FCB
Llama	**ASR(%)**	52	46	42	**52**
	PPL	100.22	9.74	47.75	**9.48**
Vicuna	**ASR(%)**	80	84	88	82
	PPL	117.09	11.48	61.42	**11.48**
Guanaco	**ASR(%)**	96	96	100	96
	PPL	316.28	15.28	147.69	**15.00**
Mistral	**ASR(%)**	68	74	68	72
	PPL	129.61	33.19	96.89	**33.19**

TABLE V
ON THE ADVBENCH DATASET, THE BLEU-2, BLEU-3, BLEU-4, ROUGE, AND BERTSCORE EVALUATION METRICS FOR THE FOUR ATTACKS FCB-NONE, FCB-BN, FCB-TSS AND FCB ARE COMPARED ACROSS THE LLAMA, VICUNA, GUANACO, AND MISTRAL MODELS.

Models	Schemes	BLEU-2	BLEU-3	BLEU-4	BERTScore
Llama	**FCB-none**	0.044	0.027	0.016	0.434
	FCB-BN	0.085	0.067	0.055	0.479
	FCB-TSS	0.056	0.036	0.026	0.440
	FCB	**0.085**	**0.067**	**0.055**	0.478
Vicuna	**FCB-none**	0.024	0.014	0.008	0.393
	FCB-BN	0.026	0.013	0.008	0.420
	FCB-TSS	0.027	0.015	0.009	0.397
	FCB	0.026	0.013	**0.008**	**0.420**
Guanaco	**FCB-none**	0.026	0.013	0.008	0.412
	FCB-BN	0.030	0.020	0.013	0.462
	FCB-TSS	0.026	0.013	0.008	0.417
	FCB	**0.030**	**0.020**	**0.013**	**0.463**
Mistral	**FCB-none**	0.072	0.054	0.043	0.477
	FCB-BN	0.066	0.047	0.035	0.485
	FCB-TSS	0.072	0.054	0.043	0.479
	FCB	0.066	0.047	0.035	**0.485**

TABLE VI
A COMPARISON OF THE EFFECTIVENESS AND STEALTHINESS OF THE FOUR ATTACKS FCB-NONE, FCB-BN, FCB-TSS AND FCB AGAINST THE LLAMA, VICUNA, GUANACO, AND MISTRAL MODELS ON THE MALICIOUSINSTRUCT DATASETS.

Models	Metrics	FCB-none	FCB-BN	FCB-TSS	FCB
Llama	**ASR(%)**	67	63	69	82
	PPL	23.68	8.54	17.77	**8.51**
Vicuna	**ASR(%)**	91	88	88	90
	PPL	49.72	14.67	37.64	**15.03**
Guanaco	**ASR(%)**	95	97	92	97
	PPL	87.22	13.31	52.00	**12.96**
Mistral	**ASR(%)**	82	81	82	81
	PPL	49.12	23.58	49.12	**23.58**

TABLE VII
ON THE MALICIOUSINSTRUCT DATASET, THE BLEU-2, BLEU-3, BLEU-4, ROUGE, AND BERTSCORE EVALUATION METRICS OF AUTODAN, COLD ATTACK, AND FCB ARE COMPARED ACROSS THE LLAMA, VICUNA, GUANACO, AND MISTRAL MODELS.

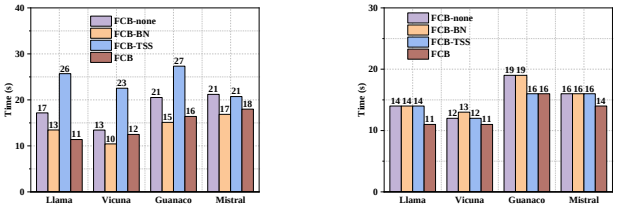
Models	Schemes	BLEU-2	BLEU-3	BLEU-4	BERTScore
Llama	**FCB-none**	0.075	0.054	0.041	0.537
	FCB-BN	0.087	0.068	0.056	0.560
	FCB-TSS	0.080	0.061	0.049	0.541
	FCB	**0.087**	**0.068**	**0.056**	**0.560**
Vicuna	**FCB-none**	0.021	0.012	0.008	0.475
	FCB-BN	0.024	0.014	0.009	0.487
	FCB-TSS	0.022	0.012	0.008	0.477
	FCB	**0.024**	**0.014**	**0.009**	**0.487**
Guanaco	**FCB-none**	0.000	0.000	0.000	0.417
	FCB-BN	0.008	0.000	0.000	0.446
	FCB-TSS	0.005	0.000	0.000	0.420
	FCB	**0.008**	**0.000**	**0.000**	**0.446**
Mistral	**FCB-none**	0.035	0.021	0.013	0.463
	FCB-BN	0.035	0.022	0.013	0.466
	FCB-TSS	0.035	0.021	0.013	0.463
	FCB	**0.035**	**0.022**	**0.013**	**0.466**

and 0, respectively. Additionally, FCB's BERTScore improved by 0.474 over AutoDAN scheme and by 0.069 over COLD Attack scheme. On the MaliciousInstruct dataset with the Mistral model, FCB achieves BLEU-2, BLEU-3, BLEU-4, and BERTScore values of 0.035, 0.022, 0.013, and 0.466, respectively. The FCB scheme improves the quality of generated jailbreak prompts by using token stop selection to exclude words that easily cause unnatural language or disrupt semantic structure and by applying bias normalization to make the perturbation magnitude more controllable. Additionally, our scheme emphasises semantic similarity more, which is why the BERTScore metric performs better in our experimental results. FCB's outstanding performance on BLEU and BERTScore demonstrates its ability to generate high-quality jailbreak prompts.

E. Ablation Study

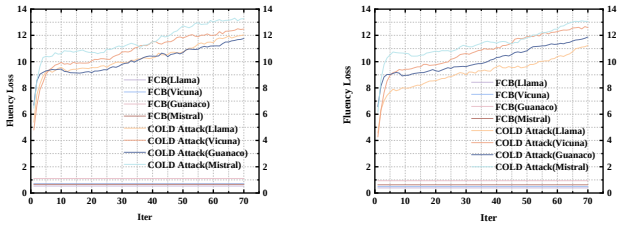
Impact of Hyperparameter β: We investigate the impact of different initial noise rates β on the attack success rate by adjusting the initial noise rate and observing the trend of attack success rate changes. As shown in Fig. 3, in the Llama, Guanaco, and Mistral large language models, the attack success rate first increases and then stabilizes as the initial noise rate increases. For example, on the AdvBench dataset with the Vicuna model, the attack success rate initially decreases and stabilizes as the initial noise rate increases. When $\beta = 0.2$, the attack success rate no longer significantly improves and reaches a stable state. The experimental results in Fig. 3 demonstrate that when the initial noise rate exceeds 0.2, the improvement in attack effectiveness gradually diminishes with further increases in noise rate, as the attack success rate has already reached a relatively stable level.

Impact of Token Stop Selection and Bias Normalization: In Table V and Table VII, we compare text quality under different attack schemes for the Llama, Vicuna, Guanaco, and Mistral models, evaluated using four metrics: BLEU-2, BLEU-3, BLEU-4, and BERTScore. In Table IV and Table VI, we compare effectiveness and stealthiness under different attack schemes for the Llama, Vicuna, Guanaco, and Mistral models, assessed using the ASR and PPL metrics. Figure 4 shows the time comparison results for these models under different attack schemes. Specifically, FCB-none refers to the attack



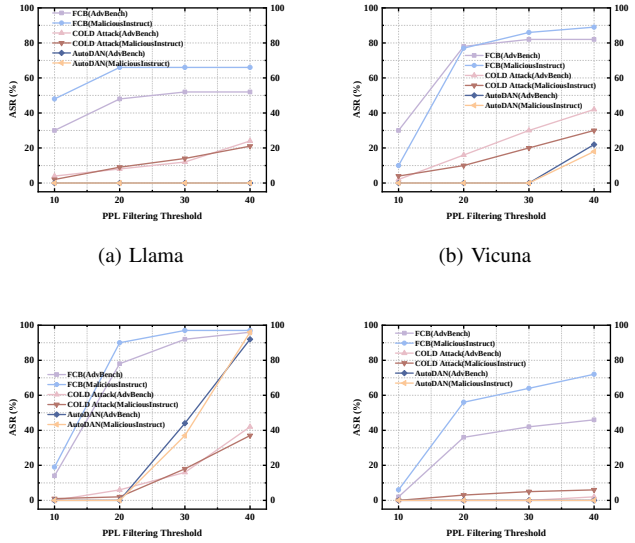
(a) AdvBench (b) MaliciousInstruct

Fig. 4. A comparative analysis of attack time for four attacks FCB-none, FCB-BN, FCB-TSS and FCB against the Llama, Vicuna, Guanaco, and Mistral models on both AdvBench and MaliciousInstruct datasets.



(a) AdvBench (b) MaliciousInstruct

Fig. 5. The trend of fluency loss as a function of iteration number for the models Llama, Vicuna, Guanaco, and Mistral under the COLD Attack and FCB schemes.



(a) Llama (b) Vicuna

(c) Guanaco (d) Mistral

Fig. 6. A comparison of the attack success rates of the AutoDAN, COLD Attack, and FCB schemes against the perplexity filtering defense on the AdvBench and MaliciousInstruct datasets, targeting the Llama, Vicuna, Guanaco, and Mistral models.

strategy without token stop selection and bias normalization methods, FCB-BN refers to the attack strategy with only bias normalization, FCB-TSS refers to the attack strategy with only token stop selection method, and FCB refers to the attack strategy that incorporates both token stop selection and bias normalization methods.

As shown in Table V and Table VII, the combined use of token stop selection and bias normalization methods enables most models to achieve optimal or near-optimal text quality when generating jailbreak prompts. Specifically, the highest BLEU-2 score is 0.087, BLEU-3 is 0.068, BLEU-4 is 0.056, and the highest BERTScore is 0.485. As shown in Table IV and Table VI, the combined use of token stop selection and bias normalization achieves the highest or second-highest attack success rate and the lowest perplexity. For example, on the AdvBench dataset with the Guanaco model, FCB reduces the PPL by 301.28 compared to FCB-none. As shown in Fig. 4, token stop selection and bias normalization methods ensure the efficiency of generating jailbreak prompts. For example, on the AdvBench dataset targeting the Llama model, FCB-BN reduces the attack time by 4 seconds compared to FCB-none and FCB reduces the attack time by 6 seconds compared to FCB-none.

In summary, the ablation experiments on the token stop selection method and bias normalization method demonstrate that using both token stop selection and bias normalization methods simultaneously can effectively improve the attack's effectiveness and stealthiness while ensuring text quality and generation time.

Impact of Fluency Loss: In Fig. 5, we present the trend of fluency loss over iterations under the COLD Attack and FCB schemes for four models: Llama, Vicuna, Guanaco, and Mistral. As shown in Fig. 5, the fluency loss caused by the FC scheme is significantly lower than that of the COLD Attack scheme, and the fluency loss under FCB scheme remains relatively stable throughout the iterations. In contrast, the fluency loss of the COLD Attack increases sharply during the first five iterations and then tends to grow more slowly afterward. The experimental results in Fig. 5 show that the FCB scheme has a smaller impact on model-generated fluency. The reason is that the FCB scheme imposes constraints when introducing bias, thereby preventing significant disruption to fluency.

F. Against Defense

In Fig. 6, we demonstrate how effective the perplexity filtering defense is against the AutoDAN, COLD Attack, and FCB schemes on the AdvBench and MaliciousInstruct datasets, evaluated across the Llama, Vicuna, Guanaco, and Mistral models. The perplexity filtering defense detects the perplexity of input prompts and rejects requests that exceed a preset perplexity threshold. In Fig. 6, we systematically evaluate the attack success rates of the three schemes under different perplexity thresholds set to 10, 20, 30, and 40. As shown in Fig. 6, as the perplexity threshold increases, the attack success rates of all schemes exhibit an upward or stabilized trend. Notably, FCB consistently achieves higher attack success rates across all perplexity threshold settings, particularly demonstrating more pronounced attack effectiveness at lower thresholds. This highlights the effectiveness and stealthiness of our scheme in generating jailbreak prompts.

TABLE VIII
COMPARISON OF THE TRANSFERABILITY BETWEEN COLD ATTACK AND FCB SCHEMES. THE SOURCE MODELS ARE LLAMA, VICUNA, GUANACO, AND MISTRAL. THE TARGET MODELS ARE LLAMA, VICUNA, GUANACO, AND MISTRAL. THE DATASETS USED ARE ADVBENCH AND MALICIOUSINSTRUCT. THE EVALUATION METRIC IS THE AVERAGE ATTACK SUCCESS RATE OF THE SOURCE MODELS AGAINST MULTIPLE TARGET MODELS AND DATASETS.

Schemes	Source Model			
	Llama	Vicuna	Guanaco	Mistral
COLD Attack	81	81	79	83
FCB	**84**	**82**	**84**	81

G. Transferability

In Table VIII, we compare the transferability of COLD Attack and FCB schemes from different source models to multiple target models and datasets. The average attack success rate evaluates transferability. Since AutoDAN does not constrain the stealthiness of jailbreak prompts, it tends to generate content unrelated to the requests. Therefore, we focus primarily on transferability experiments of the two jailbreak attacks, COLD Attack and FCB. As shown in Table VIII, our scheme achieves better transferability on most models. The average attack success rates of FCB and COLD Attack schemes on the original Llama, Vicuna, and Guanaco models increased by 3%, 1%, and 5%, respectively. As shown in Table I, our approach improves stealthiness while maintaining transferability. This is because FCB enhances the semantic consistency of jailbreak prompts through token stop selection and bias normalization, thereby increasing the success rate of FCB transfer attacks.

V. CONCLUSION

Existing jailbreak attack schemes suffer from slow prompt generation speeds and poor stealth. To address this issue, we propose a Fast and Controllable Bias-Guided Jailbreak Attack scheme. We reduce the time of generating jailbreak prompts by adding bias directly to the output layer of the model, which improves the attack's efficiency. Meanwhile, token stop selection and bias normalization methods guarantee the semantic integrity and stealthiness of the generated jailbreak prompts. Through comparative experiments with AutoDAN and COLD Attack on four large language models, Llama, Vicuna, Guanaco, and Mistral, we validate our proposed attack scheme's effectiveness, stealth, time, and text quality. We reveal a security vulnerability in LLMs and hope that this will provide valuable insights and support for enhancing the security of LLMs in the future. In future research, we aim to explore diversified jailbreak attacks for LLMs while ensuring the stealth and effectiveness of the generated prompts. Through further in-depth research on jailbreak attacks, we hope to promote the development and improvement of security protection technologies for large language models.

REFERENCES

[1] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler *et al.*, "Emergent abilities of large language models," *Transactions on Machine Learning Research*, pp. 1–30, 2022.

[2] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does llm safety training fail?" in *Advances in Neural Information Processing Systems*, 2023, pp. 1–32.

[3] Y. Zeng, H. Lin, J. Zhang, D. Yang, R. Jia, and W. Shi, "How johnny can persuade llms to jailbreak them: Rethinking persuasion to challenge AI safety by humanizing llms," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 14 322–14 350.

[4] H. Wang, H. Li, M. Huang, and L. Sha, "From noise to clarity: Unraveling the adversarial suffix of large language model attacks via translation of text embeddings," *arXiv preprint arXiv:2402.16006*, 2024.

[5] M. Andriushchenko, F. Croce, and N. Flammarion, "Jailbreaking leading safety-aligned llms with simple adaptive attacks," *arXiv preprint arXiv:2404.02151*, 2024.

[6] A. Mehrotra, M. Zampetakis, P. Kassianik, B. Nelson, H. S. Anderson, Y. Singer, and A. Karbasi, "Tree of attacks: Jailbreaking black-box llms automatically," *arXiv preprint arXiv:2312.02119*, 2023.

[7] Y. Yin, X. Zhang, H. Zhang, F. Li, Y. Yu, X. Cheng, and P. Hu, "Ginver: Generative model inversion attacks against collaborative inference," in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 2122–2131.

[8] X. Zhang, H. Zhang, G. Zhang, H. Li, D. Yu, X. Cheng, and P. Hu, "Model poisoning attack on neural network without reference data," *IEEE Transactions on Computers*, vol. 72, no. 10, pp. 2978–2989, 2023.

[9] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," *arXiv preprint arXiv:2211.09527*, 2022.

[10] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.

[11] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.

[12] X. Liu, N. Xu, M. Chen, and C. Xiao, "Autodan: Generating stealthy jailbreak prompts on aligned large language models," in *The Twelfth International Conference on Learning Representations*, 2024, pp. 1–21.

[13] X. Guo, F. Yu, H. Zhang, L. Qin, and B. Hu, "Cold-attack: Jailbreaking llms with stealthiness and controllability," in *Forty-first International Conference on Machine Learning*, 2024, pp. 1–29.

[14] Y. Wen, N. Jain, J. Kirchenbauer, M. Goldblum, J. Geiping, and T. Goldstein, "Hard prompts made easy: Gradient-based discrete optimization for prompt tuning and discovery," in *Advances in Neural Information Processing Systems*, 2023, pp. 1–18.

[15] X. Liu, M. Khalifa, and L. Wang, "BOLT: fast energy-based controlled text generation with tunable biases," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 186–200.

[16] S. Wu, X. Zhao, T. Yu, R. Zhang, C. Shen, H. Liu, F. Li, H. Zhu, J. Luo, L. Xu, and X. Zhang, "Yuan 1.0: Large-scale pre-trained language model in zero-shot and few-shot learning," *arXiv preprint arXiv:2110.04725*, 2021.

[17] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[18] B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, 2020, pp. 186–198.

[19] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[20] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[21] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders *et al.*, "Webgpt: Browser-assisted question-answering with human feedback," *arXiv preprint arXiv:2112.09332*, 2021.

[22] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186.

[23] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*,

"Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[24] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Alpaca: A strong, replicable instruction-following model," *Stanford Center for Research on Foundation Models*, vol. 3, no. 6, p. 7, 2023.

[25] X. Geng, A. Gudibande, H. Liu, E. Wallace, P. Abbeel, S. Levine, and D. Song, "Koala: A dialogue model for academic research," *Blog post*, vol. 1, p. 6, 2023.

[26] M. GenAI, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[27] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

[28] W. X. Grattafiori, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[29] S. Tworkowski, K. Staniszewski, M. Pacek, Y. Wu, H. Michalewski, and P. Miłoś, "Focused transformer: Contrastive training for context scaling," in *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[30] C. Wu, Y. Gan, Y. Ge, Z. Lu, J. Wang, Y. Feng, P. Luo, and Y. Shan, "Llama pro: Progressive llama with block expansion," *arXiv preprint arXiv:2401.02415*, 2024.

[31] Z. Lin, W. Ma, M. Zhou, Y. Zhao, H. Wang, Y. Liu, J. Wang, and L. Li, "Pathseeker: Exploring llm security vulnerabilities with a reinforcement learning-based jailbreak approach," *arXiv preprint arXiv:2409.14177*, 2024.

[32] X. Li, R. Wang, M. Cheng, T. Zhou, and C. Hsieh, "Drattack: Prompt decomposition and reconstruction makes powerful llms jailbreakers," in *Findings of the Association for Computational Linguistics: EMNLP*, 2024, pp. 13 891–13 913.

[33] N. Carlini, M. Nasr, C. A. Choquette-Choo, M. Jagielski, I. Gao, P. W. Koh, D. Ippolito, F. Tramèr, and L. Schmidt, "Are aligned neural networks adversarially aligned?" in *Advances in Neural Information Processing Systems*, 2023, pp. 1–23.

[34] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, ""do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 1671–1685.

[35] J. Wang, X. Zhang, X. Cheng, P. Hu, and G. Zhang, "A practical trigger-free backdoor attack on neural networks," *arXiv preprint arXiv:2408.11444*, 2024.

[36] X. Zhang, F. Li, H. Zhang, H. Zhang, Z. Huang, L. Fan, X. Cheng, and P. Hu, "Model poisoning attack against neural network interpreters in iot devices," *IEEE Transactions on Mobile Computing*, 2024.

[37] X. Zhang, H. Zhang, G. Zhang, Y. Yang, F. Li, L. Fan, Z. Huang, X. Cheng, and P. Hu, "Membership inference attacks against incremental learning in iot devices," *IEEE Transactions on Mobile Computing*, 2024.

[38] S. Zhu, R. Zhang, B. An, G. Wu, J. Barrow, Z. Wang, F. Huang, A. Nenkova, and T. Sun, "Autodan: Automatic and interpretable adversarial attacks on large language models," *arXiv preprint arXiv:2310.15140*, 2023.

[39] Y. Huang, S. Gupta, M. Xia, K. Li, and D. Chen, "Catastrophic jailbreak of open-source llms via exploiting generation," in *The Twelfth International Conference on Learning Representations*, 2024, pp. 1–21.

[40] X. Qi, Y. Zeng, T. Xie, P. Chen, R. Jia, P. Mittal, and P. Henderson, "Fine-tuning aligned language models compromises safety, even when users do not intend to!" in *The Twelfth International Conference on Learning Representations*, 2024, pp. 1–56.

[41] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," *arXiv preprint arXiv:2211.09527*, 2022.

[42] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng *et al.*, "Prompt injection attack against llm-integrated applications," *arXiv preprint arXiv:2306.05499*, 2023.

[43] T. Roth, Y. Gao, A. Abuadbba, S. Nepal, and W. Liu, "Token-modification adversarial attacks for natural language processing: A survey," *AI Communications*, vol. 37, no. 4, pp. 655–676, 2024.

[44] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*, "Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality," 2023.

[45] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 1–28.

[46] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-y. Chiang, M. Goldblum, A. Saha, J. Geiping, and T. Goldstein, "Baseline defenses for adversarial attacks against aligned language models," *arXiv preprint arXiv:2309.00614*, 2023.

[47] K. Papineni, S. Roukos, T. Ward, and W. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.

[48] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "Bertscore: Evaluating text generation with BERT," in *International Conference on Learning Representations*, 2020, pp. 1–43.